

Learn CUTLASS the Hard Way

Hacking on GEMMs on RTX 4090 and H100

About Me

- Kapil Sharma
- Software Engineer on the PyTorch Team @ Meta

About This Talk

- Two-part deep dive into GEMM optimization using CUTLASS
- **Part 1:** Hand-rolled kernels → CUTLASS on RTX 4090 (Ada Lovelace)
- **Part 2:** Hopper architecture on H100, CUTLASS 3.x, reaching ~90% of PyTorch performance
- Blog posts: [Part 1](#), [Part 2](#)
- Code: [explore-gemm repository](#)

Agenda

Part 1: RTX 4090 (Ada)

- Hardware baseline & roofline model
- 8-stage GEMM optimization journey
- Key principles

Part 2: H100 (Hopper)

- Hopper architecture features (TBC, TMA, Async Barriers, WGMMA)
- CUTLASS 3.x and kernel implementations
- Scheduling strategies: Stream-K, Persistent
- CTA rasterization & swizzle
- Autotuning results

Part 1: Learn CUTLASS the Hard Way

RTX 4090 (Ada Lovelace)

Hardware Baseline: RTX 4090

Metric	Value
FP32 Throughput	82.6 TFLOPS
Memory Bandwidth	1,008 GB/s
Architecture	Ada Lovelace

Roofline model insight:

- Modern GEMMs transition to compute-bound at ~ 82 FLOP/byte arithmetic intensity
- Memory-bound regime: performance $\propto$ bandwidth
- Compute-bound regime: performance $\propto$ TFLOPS

Goal: maximize arithmetic intensity to stay compute-bound

8-Stage GEMM Optimization Journey

Stage	Technique	% of PyTorch
1	Naive kernel	0.76%
2	Global memory coalescing	~5.8%
3	Shared memory caching	~7.2%
4	1D block tiling	~21.6%
5	2D block tiling	38.7%
6	Vectorized memory access	~49%
7	Warp-level tiling	54.4%
8	Tensor cores + double buffering	~60%+

Stage 1: Naive Kernel

- One thread computes one output element
- Each thread accesses global memory for every multiply-add
- **0.76% of PyTorch performance**

```
// Each thread computes C[row][col]
float sum = 0.0f;
for (int k = 0; k < K; k++) {
    sum += A[row * K + k] * B[k * N + col];
}
C[row * N + col] = sum;
```

No data reuse → bandwidth-limited at extremely low efficiency

Stage 2: Global Memory Coalescing

- Restructure memory access patterns so adjacent threads access adjacent memory
- GPU memory controllers can coalesce adjacent accesses into single transactions
- **7.6x improvement** over naive

Key insight: Thread access pattern must align with memory layout

```
Naive:      Thread 0 → A[0,0], Thread 1 → A[1,0]  (non-coalesced)
Coalesced: Thread 0 → A[0,0], Thread 1 → A[0,1]  (coalesced)
```

Stage 3: Shared Memory Caching

- Load tiles of A and B into on-chip shared memory
- Threads within a block reuse cached data
- Shared memory: ~100x lower latency than global memory
- **1.24x improvement** (modest — still limited by compute efficiency)

```
__shared__ float As[TILE][TILE];  
__shared__ float Bs[TILE][TILE];  
  
// Load tile into shared memory  
As[threadIdx.y][threadIdx.x] = A[...];  
Bs[threadIdx.y][threadIdx.x] = B[...];  
__syncthreads();
```

Stage 4 & 5: Block Tiling (1D and 2D)

1D Block Tiling: Each thread computes multiple output elements along one dimension

- Register-level computation reuse
- **3x speedup** over stage 3

2D Block Tiling: Bidirectional register reuse

- Each thread computes a $T_M \times T_N$ register tile
- Load once, compute many times
- **38.7% of PyTorch performance**

Arithmetic intensity: $2 * T_M * T_N * BK / (T_M * BK + T_N * BK)$

Stage 6 & 7: Vectorized Access + Warp Tiling

Vectorized Memory Access:

- Use `float4` loads (128-bit) instead of scalar loads
- 4 elements per instruction → reduces instruction count
- **1.27x improvement** at large matrix sizes

Warp-Level Tiling:

- Align tile hierarchy with hardware warp structure (32 threads)
- Better occupancy management
- **54.4% of PyTorch performance**

Stage 8: Tensor Cores + Double Buffering

WMMA (Warp Matrix Multiply Accumulate):

- Hardware tensor core instructions via `nvcuda::wmma`
- Massive throughput for $16 \times 16 \times 16$ matrix tiles

Double Buffering / Software Pipelining:

- Producer-consumer pattern
- Load next tile into buffer B while computing on buffer A
- Hides memory latency behind computation
- Overlaps async memory operations with CUDA cores

Key Optimization Principles

1. **Memory hierarchy:** Global → Shared → Registers
2. **Arithmetic intensity:** Compute more outputs per loaded element
3. **Coalescing:** Align access patterns with memory layout
4. **Occupancy:** Balance threads, registers, and shared memory
5. **Pipelining:** Overlap memory transfers with computation
6. **Hardware alignment:** Match tile sizes to warp/tensor core boundaries

“ Understanding the underlying hardware mechanisms proves essential before abstracting them away through CUTLASS's template infrastructure ”

Part 2: Learn CUTLASS the Hard Way

H100 (Hopper Architecture)

Ada Baseline on H100

Before writing Hopper-specific kernels, test Ada-optimized CUTLASS kernel on H100:

- **Small matrices:** Memory-bound, reasonable performance
- **Medium matrices:** Performance peaks at ~300 TFLOPs
- **Large matrices:** Hand-written kernel plateaus at ~400 TFLOPs while PyTorch scales to 700+ TFLOPs

Why? The Ada-optimized kernel doesn't leverage Hopper's architectural improvements:

- No TMA (Tensor Memory Accelerator)
- No Thread Block Clusters
- No WGMMMA (Warpgroup-level MMA)

Hopper: New Hardware Hierarchy

```
GPU
├─ SM (Streaming Multiprocessor)
│   └─ Thread Block Cluster (NEW in Hopper!)
│       └─ Thread Block (CTA)
│           └─ Warp Group (4 contiguous warps = 128 threads)
│               └─ Warp (32 threads)
│                   └─ Thread
```

H100 SXM5: **132 SMs**, 80 GB HBM3, 3.35 TB/s bandwidth

Hopper Feature 1: Thread Block Clusters (TBC)

New hierarchy level **above thread blocks**:

- Groups of up to **8 thread blocks** that can cooperate
- Distributed shared memory: direct read/write across cluster blocks
- Better data reuse without going through global memory

Benefits:

- Cooperation across multiple SMs
- Larger effective shared-memory pools
- New programming pattern: partition work across TBCs

```
// Launch with cluster shape 1x2x1 (2 TBs cooperating)
ClusterShape = Shape<_1, _2, _1>;
```

Hopper Feature 2: Tensor Memory Accelerator (TMA)

One of Hopper's **biggest architectural changes**:

Feature	A100 (LDGSTS)	H100 (TMA)
Data path	Global → Registers → Shared	Global → Shared (direct)
Bounds checking	Manual	Automatic
Zero-out	Manual	Built-in
Async model	<code>cp.async</code>	Barrier-based

- Only **one thread per warp** issues `cuda::memcpy_async`
- Others wait on barriers, or do useful work
- Dramatically reduces register file pressure

Hopper Feature 3: Asynchronous Transaction Barriers

Enhanced from Ampere, critical for producer-consumer patterns:

```
Producer thread:  
  [Issue TMA load]————→ [Arrive with byte count]—→ [Continue working]  
  
Consumer thread:  
  [Wait on barrier]—→ [Barrier met when: all threads arrived AND  
                      all bytes transferred]—→ [Execute WGMMMA]
```

- **Arrive:** Signal completion, continue immediately
- **Wait:** Block only when results are actually needed
- Waiting threads can **sleep** (not spin) → reduces wasted cycles
- Enables full overlap of memory operations with computation

Hopper Feature 4: Warpgroup-Level MMA (WGMMMA)

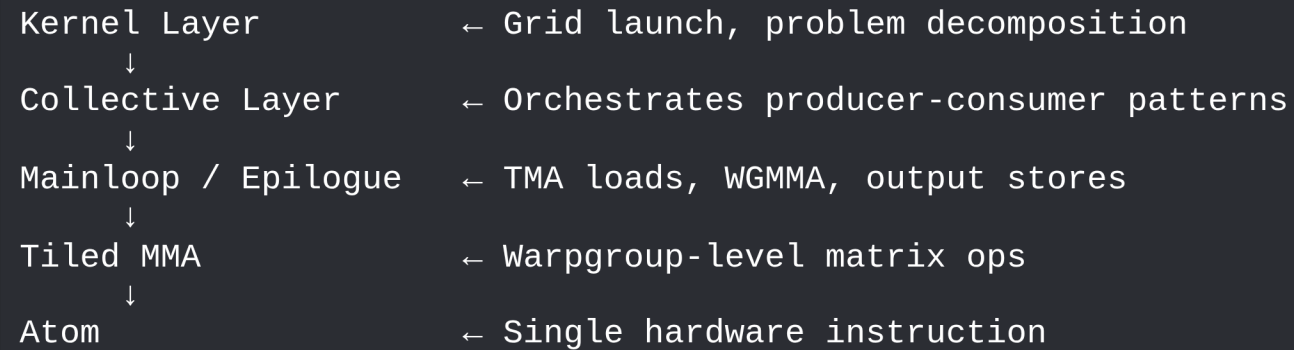
Warpgroup: 4 contiguous warps = 128 contiguous threads

```
// wgmma.mma_async executed collectively by all 128 threads  
// Supported shapes for bf16 dense:  
// m64n8k16 through m64n256k16  
wgmma::mma_async<...>(D, A_desc, B_desc, scale_d, scale_a, scale_b);
```

- First warp rank must be multiple of 4
- Much higher throughput than per-warp WMMA
- Native **FP8 support** (E4M3 and E5M2 formats)
- H100 also has 50 MB L2 cache (vs 40 MB on A100)

CUTLASS 3.x: Five-Layer Hierarchy

Redesigned for Hopper+ architectures:



Warp Specialization (Spatial Partitioning):

- Producer warps: issue TMA loads
- Consumer warps: execute WGMMMA operations
- Reduces register/shared memory pressure per warp group

Kernel Implementations on H100

Kernel 1: TMA Warp Specialized (Naive)

```
using ElementA = bfloat16_t;  
using ElementB = bfloat16_t;  
using ElementC = float;  
using TileShape = Shape<_128, _128, _64>;  
using ClusterShape = Shape<_1, _1, _1>;  
using KernelSchedule = KernelTmaWarpSpecialized;  
using EpilogueSchedule = TmaWarpSpecialized;  
// Hard-coded 2 stages
```

Result: Only 20-25% of PyTorch for large matrices — **worse** than CUTLASS 2.x baseline

Why? Only 2 pipeline stages → insufficient latency hiding for TMA transfers

Kernel 2: Auto Stage Count

Change from hard-coded 2 stages to automatic:

```
// Before:  
static constexpr int StageCount = 2;  
  
// After:  
using StageCountType = cutlass::gemm::collective::StageCountAuto;
```

For 4096×4096×4096 matrices:

- TFLOPS nearly **doubled**
- L2 cache hit rate: 63% → 73%
- SM and memory throughput increased proportionally

Still limited: ClusterShape = 1×1×1 (no inter-block data sharing)

Kernel 3: Thread Block Cluster

Add cluster cooperation along K-dimension:

```
using ClusterShape = Shape<_1, _2, _1>; // 2 TBs share memory
```

- Modest **5% performance improvement**
- Thread blocks share memory across SMs
- Still hovering at 45-55% of PyTorch for batch sizes > 1024

Observation: More gains needed from scheduling, not just hardware features

Kernel 4: Warp-Specialized Persistent Cooperative

Three improvements combined:

1. Persistent Thread Blocks:

- Fixed count (e.g., 132 for H100) — one per SM
- Each processes multiple output tiles
- Amortizes kernel launch overhead

2. Cooperative Consumers:

- Two consumer warp groups split each output tile along M
- Reduces register pressure per consumer
- Enables larger tiles → higher arithmetic intensity

3. Dynamic Tile Scheduling:

- TileScheduler assigns tiles atomically
- Considers cluster geometry and SM availability

Persistent Cooperative: Configuration

```
using KernelSchedule      = KernelTmaWarpSpecializedCooperative;  
using EpilogueSchedule    = TmaWarpSpecializedCooperative;  
using TileScheduler       = PersistentScheduler;  
static constexpr int StageCount = 5; // Auto has runtime issues
```

Performance: 60-70% of PyTorch

- 480-490 TFLOPs for 4096-8192 sizes
- vs. 700-750 TFLOPs for PyTorch

Regression for small matrices → addressed by autotuning

Kernel 5: Ping-Pong Schedule

Problem with Cooperative: Both consumer groups share A/B buffers on the same output tile → tensor cores idle during epilogue (global memory stores)

Ping-Pong solution:

```
Consumer 1: [MMA tile A] ————— [Epilogue tile A]
Consumer 2:           [Epilogue tile B] — [MMA tile B]
```

- Tile scheduler assigns each consumer a **different** output tile
- Producer fills buffers alternately
- Maximizes tensor core utilization

After experimentation, ping-pong with `stage_count = 5` offered best results but showed degradation requiring further swizzling/autotuning

Scheduling Strategies

Wave Quantization Problem

Standard GEMM partitions output tiles across SMs in discrete waves:

```
H100: 132 SMs  
133 tiles → 2 full waves (same cost as 264 tiles!)  
The 133rd tile effectively halves device utilization
```

Three approaches to solve this:

1. Data-Parallel: Reduce tile size (hurts arithmetic intensity)
2. Split-K: Divide tiles along K-dimension
3. **Stream-K**: Fractional tile assignment (eliminates quantization)

Split-K Partitioning

Divide tiles along K-dimension into constant pieces:

```
128×128×128 tile → two 128×128×64 pieces
```

- Increases work units without shrinking output tile dimensions
- Preserves arithmetic intensity better than data-parallel
- Each CTA accumulates **partial results** for its output tile
- Later CTAs wait at barrier, reduce partial results (turnstile)

Trade-off: Synchronization overhead scales with K-split factor

Stream-K: Fractional Tile Assignment

9 tiles, 4 SMs → each SM gets exactly 2.25 tiles
(vs. 3 discrete waves naively)

Algorithm:

- Phase 1: Split tiles partition along K-dimension using turnstile reduction
- Phase 2: Temporal scheduling ensures early K-pieces compute before final pieces
- Minimizes barrier wait times

Advantages over Split-K:

- Eliminates quantization entirely
- Total time ≈ 2.25 work units (vs. 3 waves)
- Most original tiles remain intact → high arithmetic intensity
- Full WGMMMA instruction availability

Hybrid Stream-K

Cache problem: Stream-K's fractional assignments break CTAs synchronization
→ Different K-offsets at different times → poor L2 cache locality

Solution: Two-phase hybrid:

```
Phase 1 (Stream-K):    Process 1 full wave + partial wave  
                        All CTAs finish simultaneously ✓  
  
Phase 2 (Data-Parallel): Remaining complete tiles  
                        Adjacent output tiles processed in sync  
                        Cache locality restored ✓
```

Result: Breaking into **500+ TFLOPs** with `stage_count = 3`

CTA Rasterization and Swizzle

Why Swizzle Matters

Naive row-major tile launch:

```
Block (0,0) → Tile (0,0)   Block (0,1) → Tile (0,1)   ...
```

- Poor L2 cache reuse
- Same data reloaded repeatedly along one dimension

Swizzle remaps (blockIdx.x, blockIdx.y):

- Improves spatial and temporal locality across tiles
- Reduces shared memory bank conflicts

```
// 32-byte swizzle pattern  
swizzled_address = (row * stride) + (col ^ ((row & 7) << 2));
```

Same column now spreads across different banks → eliminates bank conflicts

Swizzle + Rasterization Results

Size	No Swizzle	With Swizzle	Best Raster
128 ³	Minimal gain	Minimal gain	Any
512 ³	Baseline	1.2x	Along N
4096 ³	Baseline	Significant	Along N + Swizzle=8
8192 ³	Baseline	Significant	Heuristic + Swizzle=8

Pattern:

- Small sizes (128³-512³): minimal benefit from swizzling
- Large sizes (4096³-8192³): aggressive swizzling + directional rasterization

Final Autotuning Results

YOLO Autotuning: 1300+ Configurations

Swept over:

- Tile shapes ($128 \times 128 \times 64$, $128 \times 256 \times 64$, ...)
- Kernel schedules (Heuristic, Cooperative, Ping-Pong, StreamK)
- Stage counts (3, 4, 5, Auto)
- Cluster shapes ($1 \times 1 \times 1$, $1 \times 2 \times 1$)
- Swizzle factors (1, 2, 4, 8)
- Rasterization (Along M, Along N, Heuristic)
- Split strategies (DataParallel, SplitK, StreamK)

Key finding: All best configs used **TMA Persistent Cooperative** (not Ping-Pong)
with ClusterShape $1 \times 1 \times 1$

Performance Results

Size	Best TFLOPs	vs PyTorch	Best Config
128 ³	0.40	157.5%	128×128×64, Heuristic, Swizzle=1
256 ³	2.98	147.9%	128×128×64, Heuristic, Swizzle=1
512 ³	20.51	127.9%	128×128×64, Along N, Swizzle=2
1024 ³	126.14	98.8%	128×128×64, Heuristic, Swizzle=2
2048 ³	497.56	100.9%	128×256×64, Along M, Swizzle=4
4096 ³	654.97	88.1%	128×256×64, Along N, Swizzle=8, SplitK
6144 ³	672.66	96.5%	128×256×64, Along N, Swizzle=1, SplitK
8192 ³	599.33	90.2%	128×256×64, Heuristic, Swizzle=8

Configuration Analysis

Tile shape trend:

- Small sizes (≤ 512): $128 \times 128 \times 64$ optimal — fits in cache, minimal waste
- Large sizes (≥ 2048): $128 \times 256 \times 64$ — larger N tile for better arithmetic intensity

Split strategy trend:

- Small-medium (128-1024): DataParallel — minimal wave quantization
- Large (4096-6144): SplitK/StreamK — SM utilization becomes critical

Rasterization trend:

- Small: Any works (minimal impact)
- Large: Directional (Along M/N) maximizes L2 cache reuse

Summary: Lessons Learned

Part 1 (Ada / RTX 4090):

- Naive → optimized GEMM is an 80× journey
- Memory hierarchy utilization is the primary lever
- Arithmetic intensity is everything: compute more per byte loaded

Part 2 (Hopper / H100):

- TMA + WGMMMA are architectural game-changers
- Warp specialization enables true producer-consumer overlap
- Persistent kernels amortize overhead at large scale
- Stream-K eliminates wave quantization for irregular problem sizes
- Autotuning across 1300+ configs needed to reach 90% of PyTorch

Resources

- [Blog Part 1: Learn CUTLASS the Hard Way](#)
- [Blog Part 2: Learn CUTLASS the Hard Way - Part 2](#)
- [explore-gemm repository](#)
- [CUTLASS 3.x Documentation](#)
- [Hopper Architecture Deep-Dive \(NVIDIA\)](#)
- [Stream-K Paper](#)
- [Colfax Research: Persistent Kernels](#)

Questions?

Get in touch

- Discord: @sk4301
- LinkedIn: <https://www.linkedin.com/in/sharma-k/>
- Twitter: @kapil_sh_
- Github: <https://github.com/kapilsh>